

Technical Assignment — AI-Powered Form Builder

1. The Task

Build a working **AI-powered Form Builder** using Laravel, Livewire and MySQL — with manual form creation, AI generation from a prompt, and import from Word/Excel. Parts of this brief are deliberately under-specified. Those gaps are where we learn the most about you, so treat them as an invitation rather than an oversight.

2. Stack

Required: PHP 8.2+, Laravel 10/11, Livewire and/or React, MySQL 8 (migrations + seeders), Blade, JS (ES6+), Bootstrap or Tailwind, Git with a readable commit history.

Positive signals: Redis, Queues/Horizon, a separate Python FastAPI service for the AI layer consumed by Laravel over REST, Pest/PHPUnit, Docker, multi-tenant scoping. Any LLM provider is fine (free tiers included). Never commit API keys — ship a `.env.example`.

3. Part A — Core Form Builder (Mandatory)

- Add fields by **drag & drop** and **click-to-add**; reorder, duplicate, edit inline, delete. At least ten field types (text, textarea, number, email, phone, date, dropdown, radio, checkbox, file upload, section heading, rating...), groupable into sections or steps.
- Per-field config: label, key, placeholder, help text, default, required flag, options management, and validation rules (min/max, length, numeric, email, URL, regex, file type and size).
- A **JSON schema is the single source of truth** — expose it in a raw editor with two-way sync against the canvas, and validate before save.
- Clean MySQL schema with the indexes you'd actually need at scale. Mention them in the README.
- Every form gets a **public fill URL**. Server-side validation is derived from the same schema — never trust the browser. Store submissions, list them with pagination and search, allow CSV export.

4. Part B — AI Form Generation (Mandatory)

- Turn a natural-language prompt (e.g. "internship application with education history, skills and resume upload") into a complete, fully editable form — sensible types, labels, placeholders, options and validations.
- Output must be **schema-valid**. Handle malformed or partial JSON gracefully: validate, repair or retry, and never persist a broken schema.
- Support **AI editing of an existing form**, not just creation from zero — "add an emergency contact section", "make phone required", "translate labels to Hindi".
- Run generation as a **queued job with visible status**, or stream it. Don't block a web request on a long LLM call. Log model, tokens and latency.
- Document your **prompt strategy** in the README: system prompt, output contract, handling of hallucinated field types, retries and fallbacks.

5. Part C — Import from Word & Excel (Mandatory)

- Upload a **.docx** or **.xlsx** and get back an editable form. Word: headings become sections, questions become fields, checkbox or choice lists become options. Excel: support at least one clearly documented layout, and ideally a plain header-row sheet too.
- A **hybrid approach is encouraged** — parse deterministically first, use AI only to infer types and validations where the document is ambiguous. Explain the split.
- Show a **preview and mapping screen before committing** so the user can fix a wrongly detected field type. Queue large files, and report unparseable blocks clearly.
- Commit the sample files you tested with. We will also test with our own, so make the parser defensive.

6. Part D — Your Own Ideas (The Differentiator)

Parts A–C tell you what we asked for. Part D tells us who you are as an engineer. **Pick at least three improvements you genuinely believe make this product better, and build them.** Original ideas score highest; the examples below only set the level:

- **Product:** form versioning and rollback · conditional logic and branching · template library · completion and drop-off analytics · AI multi-language forms · field suggestions from similar past forms · embeddable widget or QR share.
- **Engineering:** multi-tenant isolation · Redis-cached compiled schemas · webhooks and a public submissions API · rate limiting and spam protection · autosave, undo/redo and concurrent-edit locking · test coverage · accessibility · CI and Docker.

For each idea, write a short entry covering **the user problem, your implementation, the trade-offs you accepted, and what you'd do with more time.** Three well-executed ideas beat ten half-finished ones.

7. Deliverables

1. **Public GitHub repo** with the full project and a meaningful commit history — a single "initial commit" is a negative signal.
2. **Live demo URL — mandatory.** Reachable and usable with zero setup on our side (Railway, Render, Hostinger, DigitalOcean, Azure/AWS free tiers all fine). Put the URL and demo credentials at the top of the README.
3. **README.md:** live URL and credentials, setup steps, env variables, architecture overview, schema/ERD summary, API endpoints, AI prompt strategy, known limitations.
4. **DECISIONS.md:** your assumptions, your Part D choices and why, trade-offs accepted, and what you'd build next with two more weeks.
5. Migrations and seeders (plus a SQL dump), your sample **.docx/.xlsx** import files, and — optional but genuinely helpful — a 3–5 minute walkthrough recording.

8. Ground Rules

1. **Prioritise A → B → C → D.** If you run out of time, ship what's complete and state clearly in the README what is unfinished and how you'd finish it. Honest scoping is respected; silent gaps are not.
2. **AI coding assistants are allowed and expected** — we use them too. The one condition: you can explain every line, every architectural choice and every trade-off in a live walkthrough.
3. Don't fork an existing form-builder product. Libraries and packages are fine — credit them.
4. Assumptions are welcome where the brief is silent. Write them down rather than waiting for clarification.

9. Form Builder Reference

Sample Project Code - [Backend Dev Assigment](#)

Form Builder

1 Details

2 Builder

3 Settings

4 Finish

Form basics

Survey form

Enter the primary details for your new data-collection form.

Form title *
e.g., Fall 2024 Registration
0/200
Public URL: https://127.0.0.1/submit/

Cancel

Next: Builder →

Guest User

Form Builder

✓ Details

2 Builder

3 Settings

4 Finish

Text input

STANDARD FIELDS

Text input

Text area

Number input

Email input

Phone input

Dropdown

Radio buttons

Checkboxes

Date picker

Title

Description

File upload

Download file

New line

Hidden field

← Back

Next: Settings →

